

Motivación y lenguaje C++

¿Por qué aprender un nuevo lenguaje si ya se Python?

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Agenda de la clase

1

¿Por qué otro lenguaje?

Mercado, empleabilidad junior con IA, programación competitiva y comparativa C++ vs Python.

2

Primeros pasos en C++

Anatomía de un programa, el proceso de compilación y cómo compilar (g++ y online).

3

Ejemplos: búsqueda lineal y binaria, órdenes de complejidad

Toma de contacto con la complejidad algorítmica.

4

E/S, tipos, clases y encapsulación

cin/cout, tipos básicos, enumerados, arrays, clases con atributos privados y métodos públicos.

5

Punteros y memoria dinámica

Direcciones, sizeof, paso de objetos por puntero, new/delete y por qué no hay recolector de basura.

6

Para profundizar (y ejercicios)

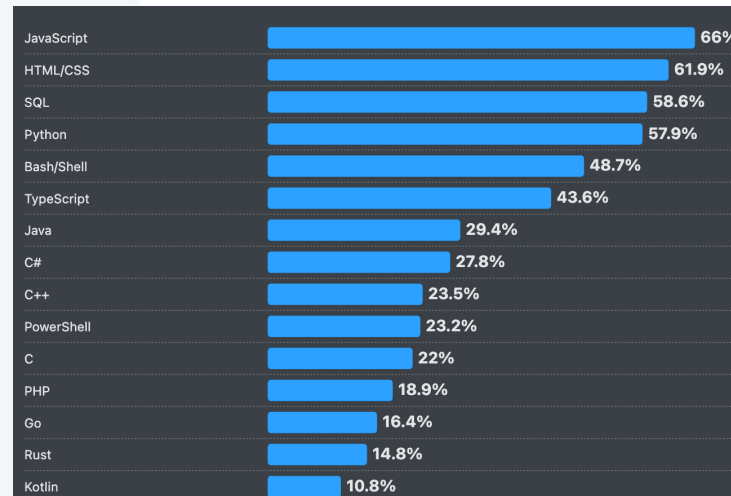
Strings, STL, templates, excepciones...

Desarrolladores

- Cuanta más gente programa en el mismo lenguaje, más difícil es diferenciarte solo con él.

57,9%

de los desarrolladores usan Python habitualmente: es el lenguaje nº1 de la encuesta, +7 puntos respecto a 2024.



Stack Overflow Developer Survey 2025
(<https://survey.stackoverflow.co>)

18,5%

cuota en el índice TIOBE (ago. 2026): Python lidera también en popularidad.

Programming Language		Ratings
	Python	18.53%
	C	11.10%
	C++	8.62%
	Java	8.25%
	C#	4.09%
	JavaScript	2.63%

TIOBE Index (<https://www.tiobe.com/tiobe-index/>),
agosto 2026.

Mercado

- Si la mayoría de candidatos junior solo sabe Python, ese perfil se vuelve menos diferencial en procesos de selección.
- Sigue siendo esencial (datos, IA, backend, scripting); el problema no es el lenguaje, es depender solo de él.
- Entender cómo funciona un lenguaje compilado y de tipado estático abre puestos a los que Python no llega.

Empleabilidad júnior

-25%

de contratación en puestos de entrada ("entry-level") en tecnología, interanual, en 2024.

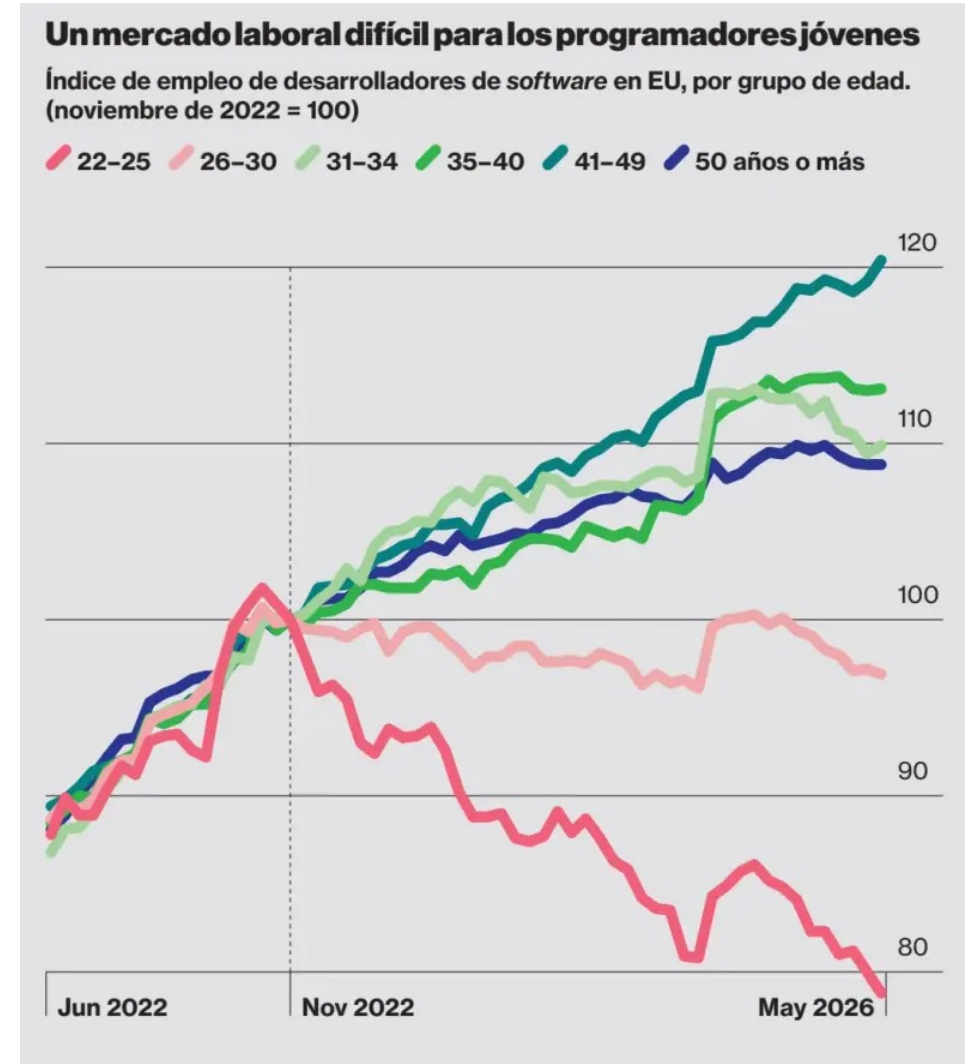
6,1% / 7,5%

tasa de paro de recién graduados en Informática / Ingeniería Informática (EE. UU.), por encima de la media.

70%

de los responsables de contratación cree que la IA ya puede hacer el trabajo de un becario.

Fuentes: análisis de mercado laboral tech EEUU (<https://stackoverflow.blog/2025/12/26/ai-vs-gen-z/> "AI vs Gen Z", dic. 2025) y SoftwareSeni, sobre datos de EE. UU.



<https://www.bloomberg.com/news/features/2026-07-22/como-chatgpt-y-claude-cambia-el-trabajo-de-los-programadores>

Qué aporta conocer C++

- Ampliar el rango de puestos a los que podéis optar:
 - Sistemas embebidos, videojuegos, robótica, simulación, drivers y sistemas operativos, computación de alto rendimiento (HPC), trading de baja latencia...
- Programar con un nivel de abstracción menor
 - C++ obliga a razonar sobre memoria, tipos y rendimiento.
- Ser un candidato más flexible
 - En un mercado más competitivo, mostrar que dominas paradigmas distintos (interpretado/compilado, dinámico/estático) pesa en el CV.
- C++ es el lenguaje dominante en los concursos
 - En ICPC, Codeforces o Google Code Jam, la inmensa mayoría de los mejores competidores programa en C++.
- La Standard Template Library (STL) como ventaja competitiva
 - Vectores, mapas, colas de prioridad, algoritmos de ordenación... ya optimizados y listos para usar.

C++ frente a Python: características clave

Característica	C++	Python
Ejecución	Compilado a código máquina antes de ejecutar	Interpretado (bytecode) en tiempo de ejecución
Tipado	Estático : el tipo se fija en compilación	Dinámico : el tipo se resuelve en ejecución
Rendimiento	Muy alto ; acceso directo a memoria y hardware	Menor por defecto (10-100x más lento en cómputo puro)
Gestión de memoria	Manual (new/delete) — sin recolector de basura	Automática , con recolector de basura (garbage collector)
Sintaxis	Llaves { }, punto y coma ;, tipos explícitos	Indentación, sin punto y coma, tipos implícitos
Detección de errores	Muchos errores se detectan al compilar	Muchos errores solo aparecen al ejecutar
Uso típico	Competición, Sistemas, videojuegos, HPC, competitiva, embebido	Datos, IA/ML, backend, automatización, prototipado

Ninguno de los dos es "mejor": son herramientas distintas para problemas distintos. Por eso interesa conocer ambos.

Compilación y ejecución

G++, flags y código máquina

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

C++ vs Python: Compilar y ejecutar

```
hola.py

print("Hola, mundo")

# Se ejecuta directamente:
# $ python hola.py
```

```
hola.cpp

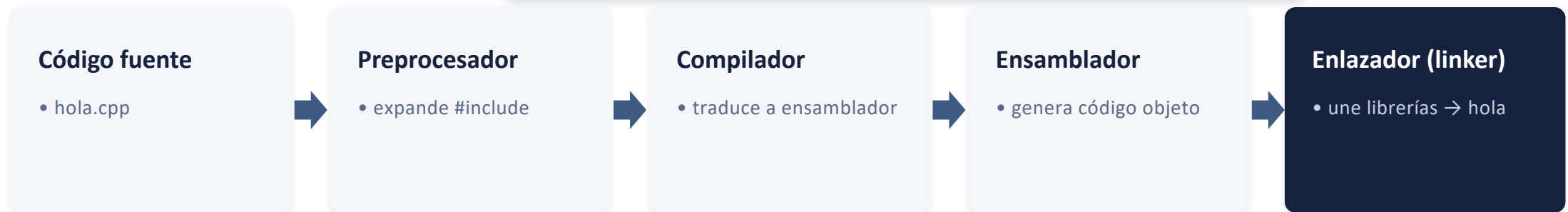
#include <iostream>
using namespace std;

int main() {
    cout << "Hola, mundo" << endl;
    return 0;
}

$ g++ -Wall -std=c++20 hola.cpp -o hola
$ ./hola
Hola, mundo
```

Por ahora podemos usar una herramienta online para compilar y ejecutar

https://www.onlinegdb.com/online_c++_compiler



Python se interpreta línea a línea; C++ se traduce entero a un ejecutable antes de correr.

Por eso un programa en C++ no se puede ejecutar hasta que compila sin errores: el compilador nos avisa de muchos fallos antes de correr nada.

Un programa en C++ y su compilación con g++

hola.cpp

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hola, mundo" << endl;
    return 0;
}
```

terminal

```
$ g++ -Wall -std=c++20 hola.cpp -o hola
$ ./hola
Hola, mundo
```

#include <iostream>

Incluye la librería para entrada/salida (cin, cout).

using namespace std;

Permite usar cout, cin... sin escribir std:: delante.

int main() { ... }

Todo programa en C++ necesita una función main, que es el punto de entrada: el sistema operativo empieza a ejecutar el programa justo ahí.

return 0;

Indica al sistema operativo que el programa terminó sin errores.

g++ El compilador de C++ de GNU

Traduce el .cpp a un programa ejecutable. Existen otros (clang++, MSVC), pero g++ es el más habitual.

Flags usados: -Wall (warnings)

Activa avisos del compilador sobre código sospechoso (variables sin usar, comparaciones extrañas, etc.).

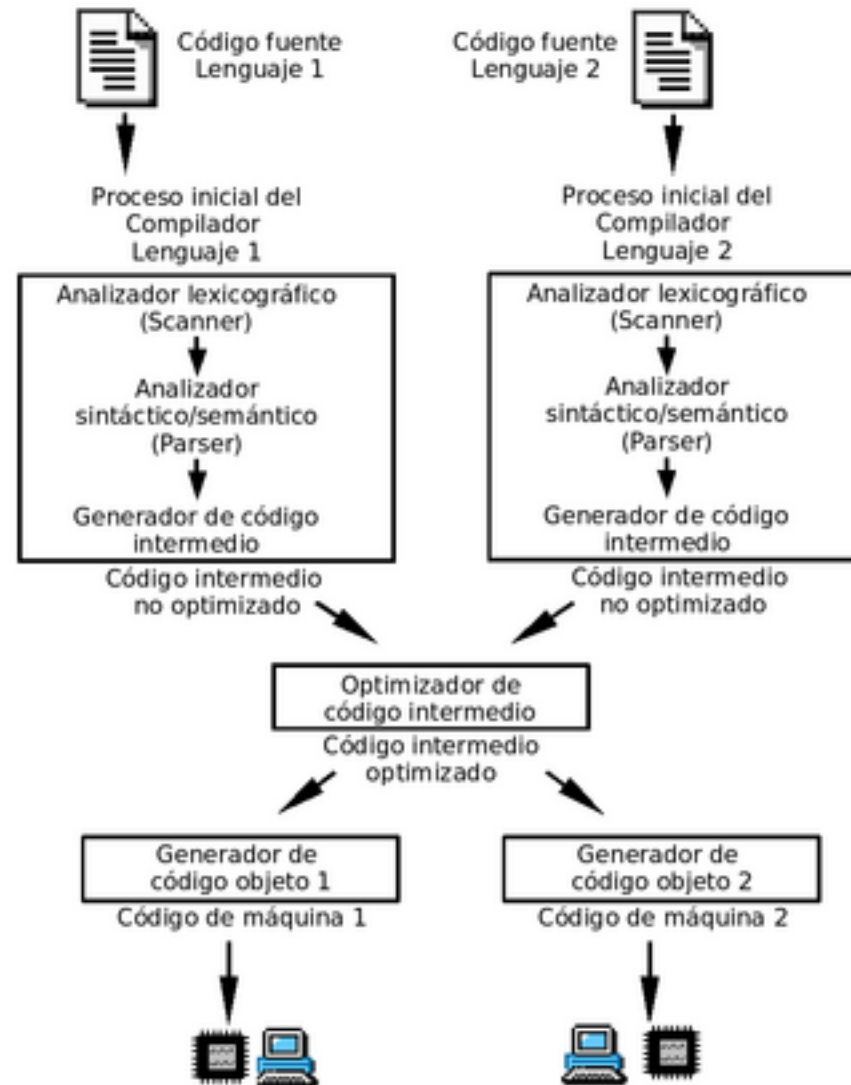
-std=c++20

Fija la versión del estándar de C++ a usar. Sin esto, el compilador puede usar una versión antigua por defecto.

-o hola

Nombre del ejecutable de salida. Sin -o, g++ genera por defecto un archivo llamado a.out.¹⁰

Compilación



Ejemplo de tareas del compilador

Imaginemos esta única línea: `resultado=x+(2*5);`

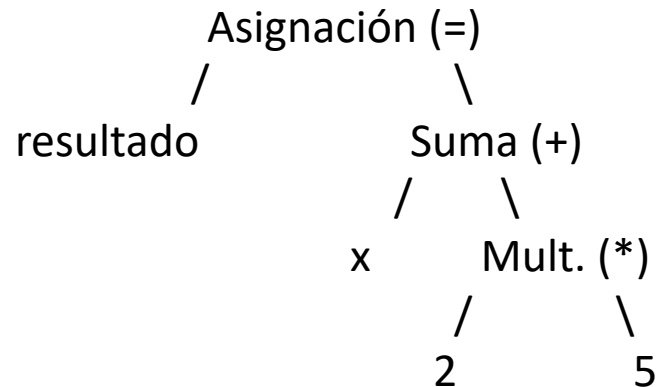
1. Análisis Léxico (Tokens)

El código se rompe en piezas mínimas.

resultado (Identificador)
= (Asignación)
x (Identificador)
+ (Operador)
((Paréntesis izquierdo)
2 (Literal numérico)
...

2. Análisis Sintáctico

Se genera el **Árbol de Sintaxis Abstracta (AST)**



3. Análisis Semántico

El compilador comprueba:
1. Existencia de variables.
2. Tipado.
3. Evalúa los tipos de las operaciones.

4. Optimización de Código

El optimizador el AST y lo simplifica ($5*2=10$)

5. Se traduce a ensamblador (código.s) (`g++ -S archivo.cpp`)
instrucciones de CPU como `ADD EAX, 10`.

6. Se pasa a código binario (código.o) (`g++ archivo.cpp`)
Se enlaza con librerías y conseguimos nuestro ejecutable. Podríamos abrirlo con `objdump -d archivo.o`

Para saber más del compilador

- <https://godbolt.org/>

The screenshot shows the Godbolt Compiler Explorer interface. On the left, the C++ source code is displayed:

```
1 // Type your code here, or load an example.
2 int square(int num) {
3     return num * num;
4 }
```

 On the right, the assembly output for x86-64 gcc (C++ trivial relocation) is shown:

```
1 "square(int)":
2     push    rbp
3     mov     rbp, rsp
4     mov     DWORD PTR [rbp-4], edi
5     mov     eax, DWORD PTR [rbp-4]
6     imul    eax, eax
7     pop     rbp
8     ret
```

imul eax, eax - Multiplica eax por sí mismo ($\text{eax} = \text{eax} * \text{eax}$). La instrucción **imul** es una multiplicación signed (con signo).

Permite ver optimizaciones: **-O2**

This screenshot shows the optimized assembly output for the same C++ code, with the optimization level set to -O2. The assembly is significantly simplified:

```
1 "square(int)":
2     imul    edi, edi
3     mov     eax, edi
4     ret
```

registros en la arquitectura x86-64:
La "r" al inicio (rbp, rsp, rdi, rax) indica que son **registros de 64 bits** (arquitectura de 64 bits). La "e" al inicio (eax, edi) indica que son la **versión de 32 bits** del mismo registro.

- **rbp** = Register Base Pointer
- **rsp** = Register Stack Pointer
- **edi** = Extended Data Index (parte baja de rdi)
- **eax** = Extended Accumulator

Nosotros usaremos C++ en VS Code

Pasos para configurar el entorno

- **Instalar el compilador:**
 - **Windows:** Instala MinGW-w64 (como MSYS2 o WinLibs) y añade su ruta bin a las variables de entorno del sistema (PATH).
 - **Linux:** Abre la terminal e instala GCC ejecutando `sudo apt install build-essential` (incluye compilador de C y C++).
 - **macOS:** Instala las herramientas de desarrollo de Xcode con el comando `xcode-select --install` en la terminal.
- **Extensiones:** Abre Visual Studio Code.
 - Ve al icono de **Extensiones** en la barra lateral izquierda (o usa `Ctrl+Shift+X`).
 - Busca e instala la extensión **C/C++** (creada por Microsoft).

Primeros ejemplos

Órdenes de complejidad de algoritmos: búsqueda en array

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Buscar un elemento en un array de números

- Enunciado: "Dado un conjunto de números (sin ningún orden particular), queremos saber si un valor está presente y, si lo está, en qué posición."

```
busqueda_lineal.py

def busqueda_lineal(datos, objetivo):
    for i, valor in enumerate(datos):
        if valor == objetivo:
            return i
    return -1

numeros = [12, 45, 3, 89, 21, 6]
print(busqueda_lineal(numeros, 89)) # 3
```

```
busqueda_lineal.cpp

#include <iostream>
using namespace std;

int busquedaLineal(const int datos[], int tam, int obj) {
    for (int i = 0; i < tam; i++) {
        if (datos[i] == obj) return i;
    }
    return -1;
}

int main() {
    int nums[] = {12, 45, 3, 89, 21, 6};
    int tam = sizeof(nums) / sizeof(nums[0]);
    cout << busquedaLineal(nums, tam, 89); // 3
    return 0;
}
```

Complejidad de la búsqueda lineal

6

vector de 6 elementos → hasta 6 comparaciones

12

vector de 12 elementos → hasta 12 comparaciones

24

vector de 24 elementos → hasta 24 comparaciones

240.000

vector de 240.000 elementos → hasta 240.000 comparaciones

$O(n)$: lineal

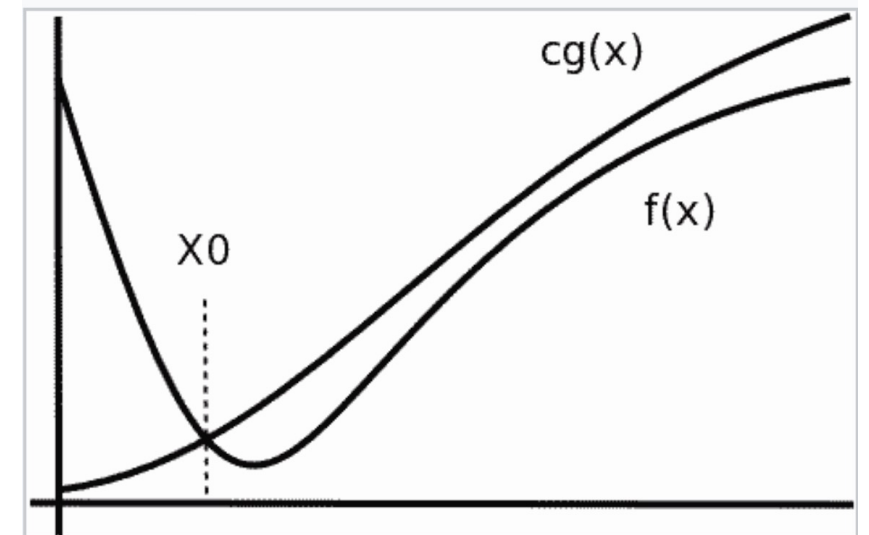
El trabajo crece en línea recta con el tamaño de la entrada (n).

Duplicar el vector duplica, **en el peor caso**, el número de comparaciones.

Qué significa $O(n)$

- Una cota superior asintótica es una función que sirve de cota superior de otra función cuando el argumento tiende a infinito.
- Usualmente se utiliza la notación de Landau: $O(g(x))$, Orden de $g(x)$, coloquialmente llamada Notación O Grande (Big-O), para referirse a las funciones acotadas superiormente por la función $g(x)$.
- Formalmente:

$$O(g(x)) = \left\{ f(x) : \text{existen } x_0, c > 0 \text{ tales que} \right. \\ \left. \forall x \geq x_0 > 0 : 0 \leq |f(x)| \leq c|g(x)| \right\}$$



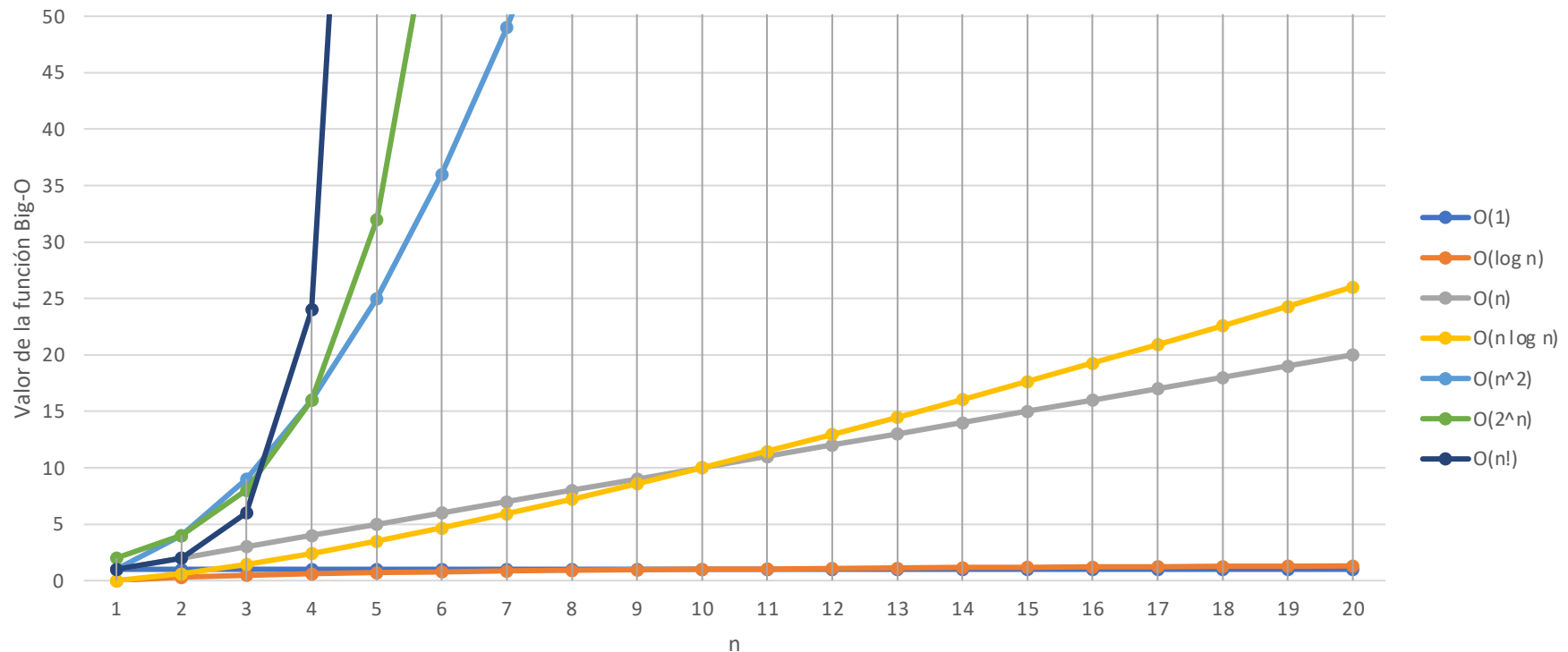
$f(x) = O(g(x))$.

¿Qué mide el orden de complejidad?

- Cómo crece el coste (tiempo, memoria) de un algoritmo o una operación de una estructura de datos (insertar, borrar, buscar...), a medida que crece el tamaño de la entrada n , sin depender de la máquina concreta.

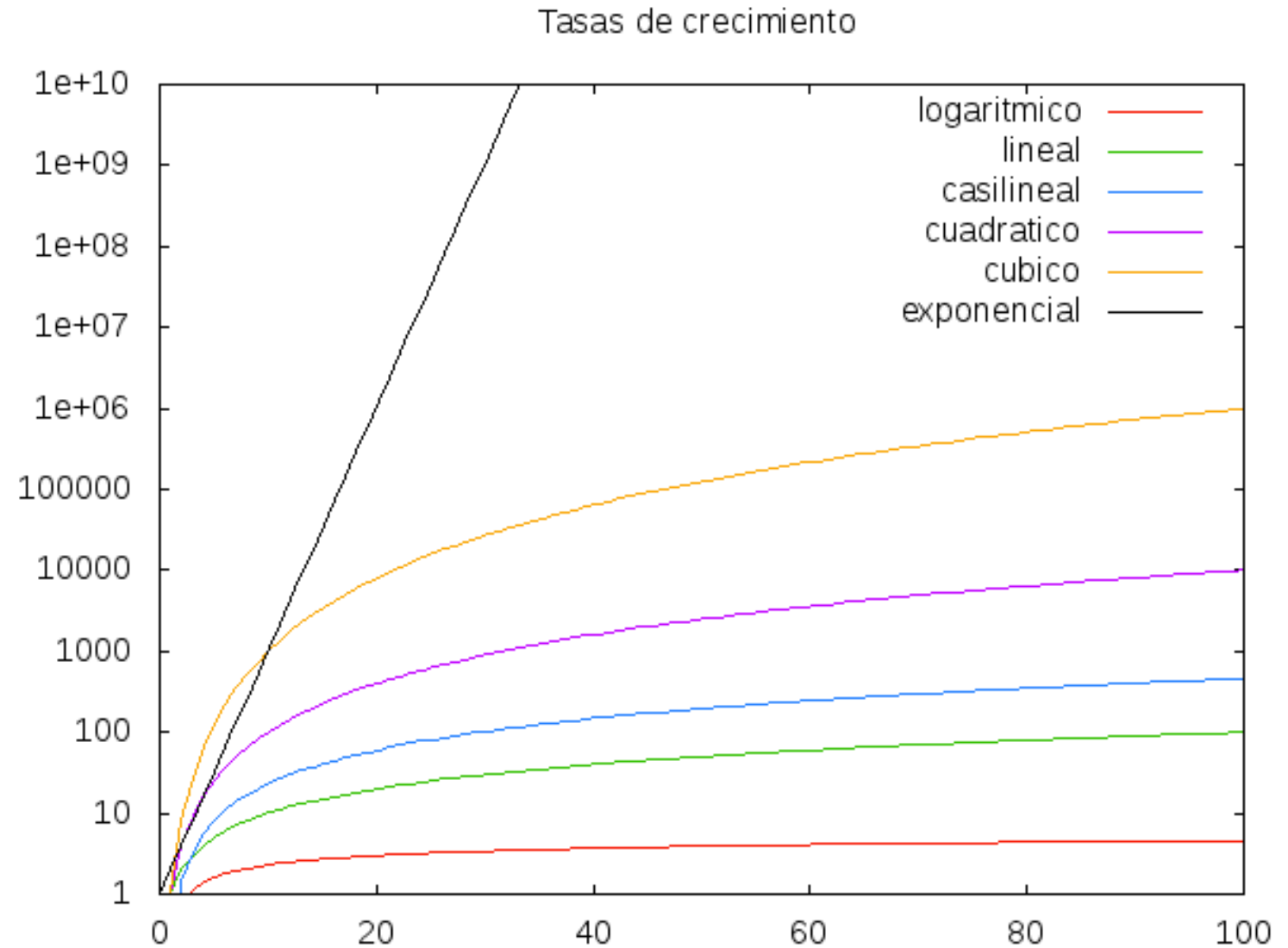
Principales órdenes de complejidad y crecimiento

Orden	Nombre
$O(1)$	constante
$O(\log n)$	logarítmica
$O(n)$	lineal
$O(n \log n)$	casi lineal
$O(n^2)$	cuadrática
$O(n^3)$	cúbica
$O(a^n)$	exponencial



Principales órdenes de complejidad y

- En escala logarítmica



Algunos ejemplos

- ¿De qué orden de complejidad son las siguientes funciones?

- $f(x) = x/2 + 4$

- $f(x) = 3x^2 + 2x + 10^6$

- $f(x) = x + \log(x)$

- $f(x) = x^3 + x^4$

- $f(x) = 3 \cdot 10^6$

Orden	Nombre
$O(1)$	constante
$O(\log n)$	logarítmica
$O(n)$	lineal
$O(n \log n)$	casi lineal
$O(n^2)$	cuadrática
$O(n^3)$	cúbica
$O(a^n)$	exponencial

- Si tuviéramos un problema resuelto por 5 métodos con esas funciones de complejidad, ¿cuál elegirías?

Array ordenado: Búsqueda binaria

busqueda_binaria.cpp

```
int busquedaBinaria(const int datos[], int tam, int obj) {  
    int izq = 0, der = tam - 1;  
    while (izq <= der) {  
        int medio = izq + (der - izq) / 2;  
        if (datos[medio] == obj) return medio;  
        else if (datos[medio] < obj) izq = medio + 1;  
        else der = medio - 1;  
    }  
    return -1;  
}  
  
int main() {  
    int nums[] = {3, 6, 12, 21, 45, 89};  
    int tam = sizeof(nums) / sizeof(nums[0]);  
    cout << busquedaBinaria(nums, tam, 89);  
    return 0;  
}
```

Paso 1



Paso 2



Paso 3 · ¡encontrado!



¿Por qué se usa $izq + (der - izq) / 2$ en vez de $(izq + der) / 2$?

Overflow

La revelación de Joshua Bloch (2006) Medium

En 1986, **Jon Bentley**, autor del clásico libro *Programming Pearls*, publicó un algoritmo de búsqueda binaria demostrado matemáticamente. En 1997, **Joshua Bloch** tomó esa misma implementación para escribir el método `java.util.Arrays.binarySearch` en el JDK de Java.

Ambas versiones usaban la fórmula ingenua:

```
int mid = (low + high) / 2;
```

En 2006 (nueve años después de entrar al JDK de Java y veinte después de ser publicado por Bentley), Joshua Bloch descubrió y admitió públicamente que el código contenía un error crítico que afectaba a casi todas las implementaciones de búsqueda binaria y *mergesort* de la época.

¿Cómo afectaba a Java y C++?

El fallo solo se manifestaba cuando la longitud del arreglo superaba 2^{30} elementos (alrededor de 1,000 millones de posiciones). Cuando la búsqueda llegaba a la mitad superior del arreglo:

Reddit

- o **En Java:** La suma `low + high` superaba `Integer.MAX_VALUE` ($2^{31} - 1$). En el sistema de complemento a dos que usa Java, el número se convertía en un **entero negativo**. Al dividirlo por 2, resultaba un índice negativo, lo que lanzaba inmediatamente la excepción `java.lang.ArrayIndexOutOfBoundsException` y destruía la ejecución del programa. Google Research
- o **En C++:** La suma de enteros con signo que sobrepasa el límite produce **Comportamiento Indefinido (Undefined Behavior)** según el estándar del lenguaje. Esto significaba que el programa podía colapsar por un fallo de segmentación (*segmentation fault*), corromper la memoria o entrar en un bucle infinito impredecible. Reddit

Se usa `izq + (der - izq) / 2` para evitar un desbordamiento de entero (overflow) en arreglos de gran tamaño.

Matemáticamente, ambas expresiones son idénticas:

$$\text{izq} + \frac{\text{der} - \text{izq}}{2} = \frac{2 \cdot \text{izq} + \text{der} - \text{izq}}{2} = \frac{\text{izq} + \text{der}}{2}$$

La diferencia radica en cómo la memoria de la computadora procesa los números en C++:

- o **El problema de `(izq + der) / 2`:** Si trabajas con un arreglo muy grande y `izq` junto con `der` son números altos (por ejemplo, cerca del límite máximo de un `int` de 32 bits, que es 2,147,483,647), la suma `izq + der` sobrepasará ese límite. Esto genera un **número negativo** por desbordamiento, lo que provocará que `medio` sea un índice negativo y el programa falle (crash) al intentar acceder a memoria no válida.
- o **La ventaja de `izq + (der - izq) / 2`:** Al restar `der - izq` primero, obtienes la distancia entre los dos índices (un número pequeño). Luego la divides entre 2 y se la sumas a `izq`. Ninguna operación intermedia supera el valor de `der`, garantizando que nunca ocurra un desbordamiento dentro del rango del tipo `int`.

```
00111011100110101100101000000000 (1,000,000,000)
+ 01110111001101011001010000000000 (2,000,000,000)
```

```
-----
10110010110100000101111000000000
(3,000,000,000 -> Overflow: -1,294,967,296)
```

Complejidad de la búsqueda binaria



3

vector de 6 elementos → hasta 3 comparaciones



4

vector de 12 elementos → hasta 4 comparaciones



5

vector de 24 elementos → hasta 5 comparaciones



18

vector de 240.000 elementos → hasta 18 comparaciones

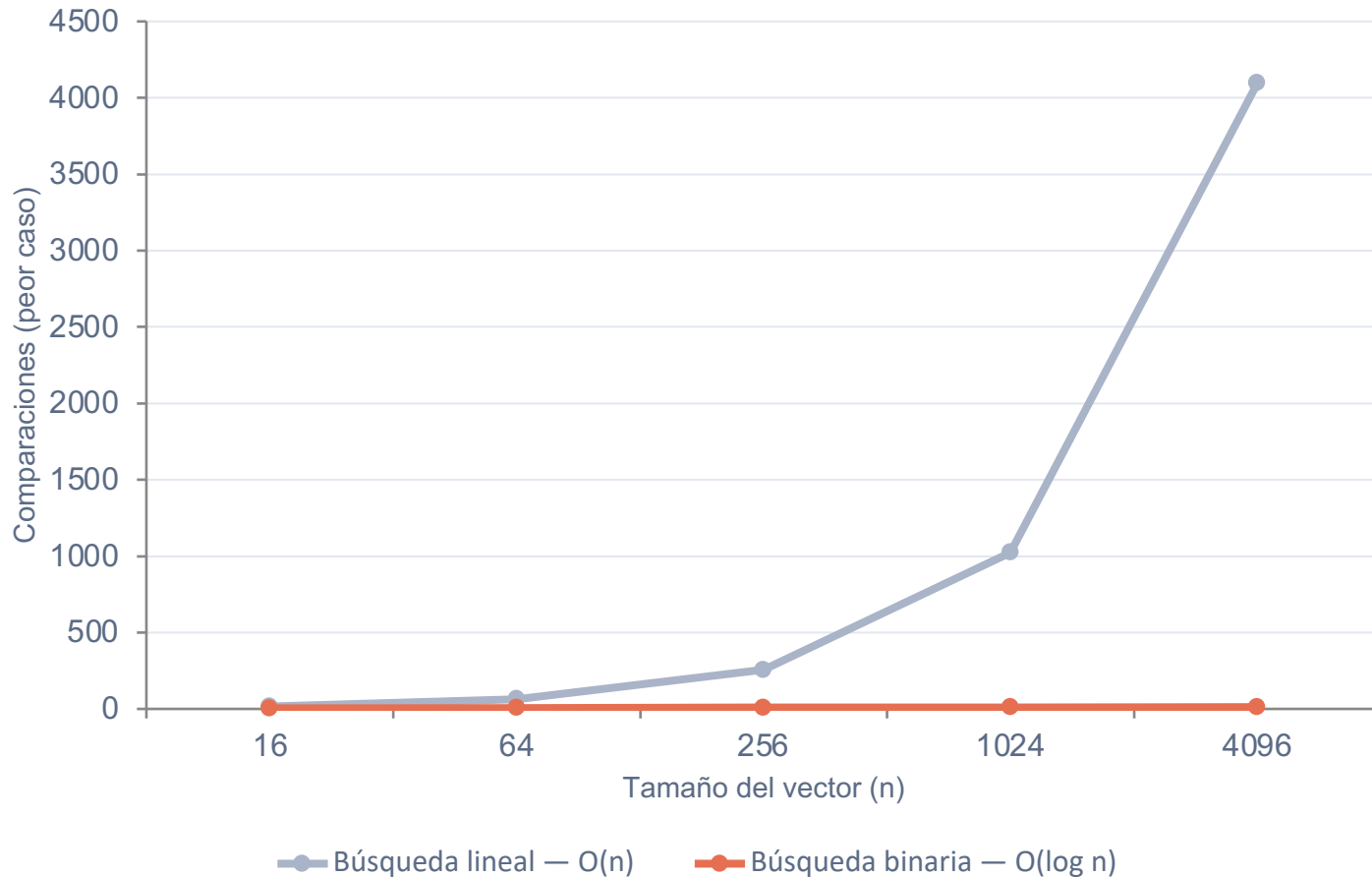
$O(\log_2 n)$:

Logarítmico

El trabajo crece
logarítmicamente con el
tamaño de la entrada (n).

Elevar al cuadrado el tamaño
del array solo duplica, en el
peor caso, el número de
comparaciones.

$O(\log n)$ frente a $O(n)$: crecimiento



Con 4096 elementos...

Lineal: hasta 4.096 comparaciones.

Binaria: solo 12 comparaciones.

Esto es lo que hace que la complejidad algorítmica importe: el mismo problema, resuelto de otra forma, puede ser miles de veces más rápido sin cambiar de lenguaje.

Órdenes de complejidad

- En AED I nos centraremos en la complejidad temporal y espacial del coste de operaciones, (como insertar, buscar o eliminar) sobre estructuras de datos:
 - ¿Qué coste supone insertar un nuevo elemento al principio de una lista con diferentes implementaciones?
 - ¿Qué complejidad tiene borrar la cima de una pila con diferentes implementaciones?
- En AED II nos centraremos en la complejidad algorítmica (tal y como hemos visto con búsqueda lineal y binaria).

C++ con ejemplos

Tipos básicos, punteros, arrays, control, funciones y clases

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Entrada/salida estándar: cin y cout

saludo.py

```
nombre = input("Nombre: ")
edad = int(input("Edad: "))

print(f"Hola {nombre}, el año que viene "
      f"tendrás {edad + 1} años.")
```

saludo.cpp

```
#include <iostream>
using namespace std;

int main() {
    string nombre;
    int edad;
    cout << "Nombre: ";
    cin >> nombre;
    cout << "Edad: ";
    cin >> edad;

    cout << "Hola " << nombre << "!" << endl;
    cout << "El proximo año tendras ";
    cout << edad + 1 << " años." << endl;
    return 0;
}
```

cin/cout usan >> y << (operadores de flujo) en vez de funciones; cin >> edad ya convierte el texto leído al tipo de la variable (int). Si no usáramos el namespace std, habría que hacer std::cin y std::cout

Tipos básicos e inicialización de variables

```
tipos.cpp

int edad = 20;           // entero
double pi = 3.14159;     // decimal doble precisión
float factor = 1.5f;     // decimal simple precisión
char inicial = 'A';      // un solo carácter
bool activo = true;      // verdadero / falso

// Tres formas válidas de inicializar:
int a = 5;               // asignación
int b(5);                // estilo constructor
int c{5};                 // inicialización uniforme (C++11)

const double GRAVEDAD = 9.81; // no se puede modificar
```

int, double, float, char, bool

Tipos fundamentales; su tamaño exacto lo veremos con sizeof en la sección de punteros.

int c{5};

La "inicialización uniforme" evita conversiones peligrosas que sí se permiten con operador de asignación "=" y paréntesis.

const

En C++ el compilador impide modificarla.

Tipos básicos

- C++ es un lenguaje de **tipado estático**: cada variable tiene un tipo fijo, decidido en tiempo de compilación, que determina cuánta memoria ocupa y qué operaciones admite.
- El estándar de C++ **no define un tamaño exacto en bytes** para la mayoría de los tipos (como int, long o long long).
- En su lugar, el estándar solo garantiza un **tamaño mínimo** y una **relación de orden** entre ellos:

`sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)`

Tipos Enumerados

```
dias.cpp

enum DiaSemana {
    LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO
};

int main() {
    DiaSemana hoy = MIERCOLES;

    if (hoy == SABADO || hoy == DOMINGO) {
        cout << "Fin de semana" << endl;
    } else {
        cout << "Día laborable" << endl;
    }
    return 0;
}
```

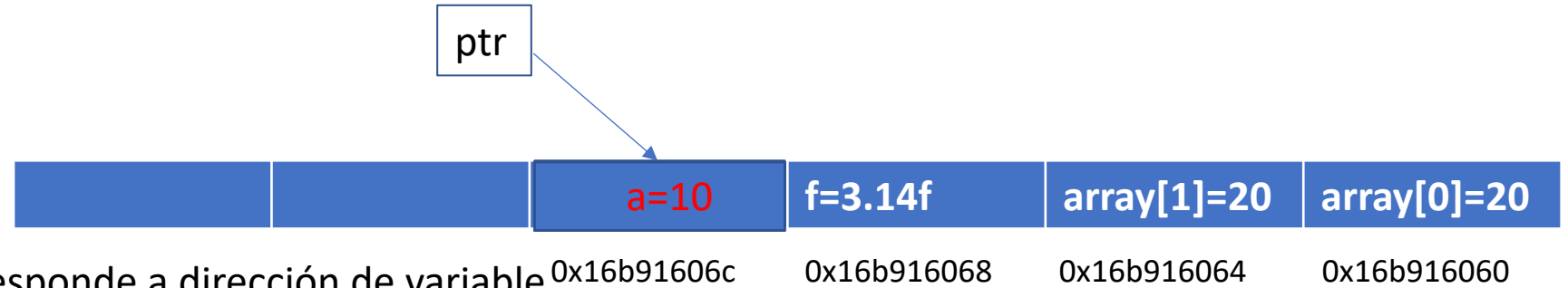
Veamos más ejemplos en Github

Punteros

- Son variables que almacenan **direcciones de memoria**, apuntando a ellas. Un puntero no puede contener números arbitrarios (la única excepción es 0/nullptr).
- En las direcciones de memoria puede haber variables, arrays, clases, otro puntero, funciones, bloque de memoria dinámica...
- Aplicaciones:
 - Acceso directo y modificación de memoria
 - Estructuras de datos enlazadas (por punteros)
 - Paso de parámetros por referencia
 - Arrays, strings, programación bajo nivel...

Punteros

```
int a=5;  
float f=3.14f;  
int array[2] = {10, 20};
```



```
int* ptr=&a; //&variable corresponde a dirección de variable  
cout << ptr; // 0x16b91606c  
*ptr=10; //operador para “desreferenciar” el puntero
```

La pila de memoria crece hacia direcciones menores

Veamos más ejemplos. Ej 03 en GitHub

UN CONCEPTO QUE
PYTHON OCULTA Y C++
HACE EXPLÍCITO

Tamaño de punteros

Tipo	sizeof(variable)	sizeof(puntero al tipo)
char	1 byte	8 bytes
int	4 bytes	8 bytes
float	4 bytes	8 bytes
double	8 bytes	8 bytes
Persona (objeto)	depende de sus atributos	8 bytes

La clave

Un puntero solo guarda una dirección de memoria: por eso su tamaño (en un sistema de 64 bits) es siempre 8 bytes, sin importar a qué tipo apunte.

int* y double* ocupan lo mismo; lo que cambia es cómo se interpretan los bytes a los que apuntan.

En Python no se ve directamente: toda variable es ya una referencia a un objeto (por eso $a = b$ nunca "copia" un objeto grande).

Ejercicio para profundizar: https://www.it.uc3m.es/pbasanta/asng/course_notes/ch05s10.html

Más material:

- <https://www.luisllamas.es/cpp-que-son-los-punteros/>
- https://youtu.be/ftLs86s6cIE?si=9_fvRR0hFJWrP5G0&t=254

Memoria dinámica: Stack frente a Heap

Stack

- Se gestiona automáticamente
- Muy rápido de reservar y liberar
- Tamaño limitado
- La variable "muere" al salir del bloque { }
- Ejemplo: `Persona p("Ana", 20);`

Heap (memoria dinámica)

- La gestiona el programador (`new / delete`)
- Más lento, pero mucho más flexible
- Tamaño solo limitado por la RAM disponible
- Vive hasta que se libera explícitamente
- Ejemplo: `Persona* p = new Persona("Ana", 20);`

DOS ZONAS DE MEMORIA MUY DISTINTAS

Memoria dinámica

```
array_dinamico.cpp

int n = 5;
int* notas = new int[n];          // reserva un array de n enteros

for (int i = 0; i < n; i++) notas[i] = i * 10;

delete[] notas;    // ¡[] obligatorio! delete notas; sería un error
notas = nullptr;
```

new[] y delete[] siempre van emparejados: usar delete (sin corchetes) sobre un array reservado con new[] es un error que puede colar sin avisar y corromper el programa.

Clases

```
class Persona
```

ATRIBUTOS (estado)

```
nombre : string
```

```
edad : int
```

MÉTODOS (comportamiento)

```
Persona(nombre, edad)
```

```
getNombre() : string
```

```
cumplirAños()
```

```
saludar()
```

1

Atributos = variables del objeto

Cada instancia (cada objeto) tiene su propia copia: dos personas distintas, dos nombres distintos.

2

Métodos = funciones del objeto

Operan sobre los atributos de esa instancia concreta (this).

3

En Python ya lo conocéis

Es exactamente el mismo concepto que class ... def __init__(self, ...): en Python.

Clases

```
persona.h

class Persona {
private:
    string nombre;
    int edad;

public:
    Persona(string n, int e) : nombre(n), edad(e) {} // constructor

    string getNombre() const { return nombre; }
    int getEdad() const { return edad; }

    void cumplirAnios() { edad++; }

    void saludar() const {
        cout << "Hola, soy " << nombre << " y tengo " << edad << " años." << endl;
    }
}
```

Persona : nombre(n), edad(e) es la "lista de inicialización" del constructor: la forma preferida en C++ de fijar los atributos al crear el objeto.

Instanciación de objetos

```
main.cpp

int main() {
    Persona p("Ana", 20); // objeto creado en el stack

    p.saludar();           // Hola, soy Ana y tengo 20 años.
    p.cumplirAnios();
    p.saludar();           // Hola, soy Ana y tengo 21 años.

    return 0;
}
```

Persona p("Ana", 20);

Se reserva espacio para el objeto

- directamente en el stack y se llama al constructor con esos argumentos.

p.saludar();

El operador . accede a métodos y atributos

- públicos, igual que en Python con objeto.metodo().

p.edad no compila

Al ser privado, ningún código fuera de la

- clase puede leerlo o modificarlo directamente: hay que usar getEdad().

¿Por qué encapsular?



Proteger el estado interno

Nadie puede dejar edad en un valor inválido (p.ej. negativo) saltándose la lógica de la clase.



Puedes cambiar la implementación sin romper nada

Si mañana guardas la edad como fecha de nacimiento, el código externo que usa `getEdad()` no se entera.



public = contrato; private = detalles

Lo público es "cómo se usa la clase"; lo privado es "cómo funciona por dentro", y puede cambiar libremente.

Clase con memoria dinámica

```
persona_dinamica.cpp

class Persona {
private:
    string nombre;
public:
    Persona(string n) : nombre(n) { cout << "[Constructor] " << nombre << endl; }
    ~Persona() { cout << "[Destructor] " << nombre << endl; }
    void saludar() const { cout << "Hola, soy " << nombre << endl; }
};

int main() {
    { Persona local("Local"); local.saludar(); } // destructor al cerrar el bloque
    Persona* p = new Persona("Dinamica"); // constructor
    p->saludar();
    delete p; // destructor -- si se olvida, memory leak
    return 0;
}
```

El destructor (~Persona) es donde se libera cualquier recurso que la clase haya reservado: es el sitio natural para "limpiar" tras un objeto.

Buenas prácticas con memoria dinámica

Fugas de memoria (memory leaks)



Reservar con `new` y no liberar nunca: la memoria queda ocupada mientras el programa siga corriendo.

Punteros colgantes (dangling pointers)



Usar un puntero después de hacer `delete` sobre lo que apuntaba: comportamiento indefinido, puede "funcionar" y fallar horas después.

Buenas prácticas de hoy



Cada `new` con su `delete`; poner el puntero a `nullptr` tras liberar; nunca usar algo después de borrarlo.

MÁS ADELANTE EN EL CURSO

C++ moderno ofrece "punteros inteligentes" (`unique_ptr`, `shared_ptr`) que automatizan gran parte de esto sin perder el control manual. Los veremos cuando dominemos `new/delete` a mano.

Punteros inteligentes (Smart pointers)

- Problemas de los punteros:
 - Somos responsables de reservar memoria con `new` y liberarla con `delete`.
 - Si se te olvidaba el `delete`: un memory leak.
 - Si lo haces dos veces: corrupción de memoria.
 - Si accedes después de borrar: segmentation fault.
- C++11 introdujo los Smart Pointers
 - Una clase que envuelve a un puntero (raw pointer) y se encarga de gestionar su ciclo de vida automáticamente.
 - Atar el ciclo de vida de un recurso (memoria, archivos, sockets) al ciclo de vida de un objeto.
 - Cuando creamos un Smart Pointer (una variable en el Stack), este adquiere el recurso (la memoria en el Heap). Cuando el Smart Pointer sale de su ámbito (scope), su destructor se ejecuta automáticamente y libera la memoria.

Punteros inteligentes (Smart pointers)



```
void procesarDatos() {  
    MiObjeto* obj = new MiObjeto(); // Reservamos memoria  
    obj->hacerAlgo();  
  
    // ... código complejo ...  
  
    if (hayError) {  
        return; // ¡OPS! Nos vamos sin hacer delete. Memory Leak.  
    }  
  
    delete obj; // Si llegamos aquí, todo bien. Si no... mal asunto.  
}
```

```
#include <memory> // Necesario para usar smart pointers  
  
void procesarDatos() {  
    std::unique_ptr<MiObjeto> obj = std::make_unique<MiObjeto>();  
  
    obj->hacerAlgo(); // Se usa igual que un puntero normal (->)  
  
    if (hayError) {  
        return; // El destructor de 'obj' salta aquí. Memoria liberada.  
    }  
  
} // El destructor de 'obj' salta aquí también. Memoria liberada.
```

<https://learn.microsoft.com/es-es/cpp/cpp/smart-pointers-modern-cpp>

<https://www.luisllamas.es/cpp-que-son-smart-pointers/>

Extra

Strings, STL, templates, excepciones

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

STL: Standard Template Library

```
stl.cpp

#include <vector>
#include <algorithm>
#include <map>

vector<int> v = {5, 3, 8, 1};
sort(v.begin(), v.end());           // {1, 3, 5, 8}
auto it = find(v.begin(), v.end(), 8); // iterador al 8

map<string, int> edades;
edades["Ana"] = 20;
edades["Luis"] = 22;
```

LA RAZÓN DE QUE C++ SEA EL
LENGUAJE DE REFERENCIA EN
PROGRAMACIÓN COMPETITIVA



Contenedores

vector, array, map, set, queue, stack, priority_queue... estructuras de datos ya implementadas y probadas.



Algoritmos

sort, find, count, reverse, accumulate... funciones genéricas listas para usar sobre cualquier contenedor.

std::string frente a str de Python

```
strings.py

nombre = "Ana"
saludo = "Hola, " + nombre + "!"

print(len(saludo))
print(saludo[0:4])
print(saludo.upper())
```

```
strings.cpp

#include <string>

string nombre = "Ana";
string saludo = "Hola, " + nombre + "!";

cout << saludo.length() << endl;
cout << saludo.substr(0, 4) << endl;
cout << saludo.size() << endl;
```

std::string es mutable y de tamaño dinámico, con una API muy similar a la de Python: length()/size(), substr(), +, ==, find()...

Arrays de objetos y atributos estáticos

arrays_objetos.cpp

```
vector<Persona> alumnos = {
    Persona("Ana", 20),
    Persona("Luis", 22),
    Persona("Eva", 21)
};

for (const auto& a : alumnos) {
    a.saludar();
}

cout << alumnos.size(); // 3
```

static.cpp

```
class Persona {
    static int contador; // compartido por TODAS
public:
    // las instancias
    Persona() { contador++; }
    static int getContador() {
        return contador;
    }
};

int Persona::contador = 0; // se define fuera

Persona a, b, c;
cout << Persona::getContador(); // 3
```

Templates (funciones genéricas)

```
templates.cpp

template <typename T>
T maximo(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << maximo(3, 7) << endl;      // 7    (T = int)
    cout << maximo(3.5, 2.1) << endl;  // 3.5  (T = double)
    return 0;
}
```

IDEA: ESCRIBIR UNA FUNCIÓN QUE SIRVA PARA VARIOS TIPOS

El compilador genera una versión distinta de maximo para cada tipo T con el que se usa: es como def maximo(a, b) en Python, pero comprobado y optimizado en compilación.

Excepciones: try / catch frente a try / except

GESTIONAR ERRORES EN TIEMPO DE EJECUCIÓN

```
excepciones.py

try:
    if edad < 0:
        raise ValueError("Edad negativa")
except ValueError as e:
    print(f"Error: {e}")
```

```
excepciones.cpp

try {
    if (edad < 0) {
        throw invalid_argument("Edad negativa");
    }
} catch (const invalid_argument& e) {
    cout << "Error: " << e.what() << endl;
}
```

throw lanza la excepción, catch la captura por tipo (aquí, invalid_argument) y e.what() da el mensaje, como str(e) en Python.

Resumen

- Motivación C++
- Qué debéis saber:
 - Características que lo diferencian de Python
 - Órdenes de complejidad
 - Punteros
 - Clases

Para casa

- Ejercicios propuestos:
 - Tamaño de `char*`, enumerado, `string` en C++.
 - ¿Algún tipo varía de tamaño según la máquina?
 - Programa para saber el día de la semana (enum) dada una fecha.
 - Modificar el código de `03_01_punteros_arrays_estructuras.cpp` para que si la mejor nota es mayor a 9.5, subirla a 10.
 - Ejecutar todos los programas del 00 al 11 (opcional el 9).
 - Resolver las cuestiones planteadas en https://www.it.uc3m.es/pbasanta/asng/course_notes/ch05s10.html

Recursos

- Standard C++: <https://isocpp.org/>
- Referencia C++: <https://es.cppreference.com/>
- The history of C++:
<https://www.youtube.com/watch?v=ll7tMxzSJ7w>
- Posts de temas avanzados:
<https://dgvergel.blogspot.com/2026/05/todos-los-posts-del-blog.html>

Anexo: Sobre uso de IA en la asignatura

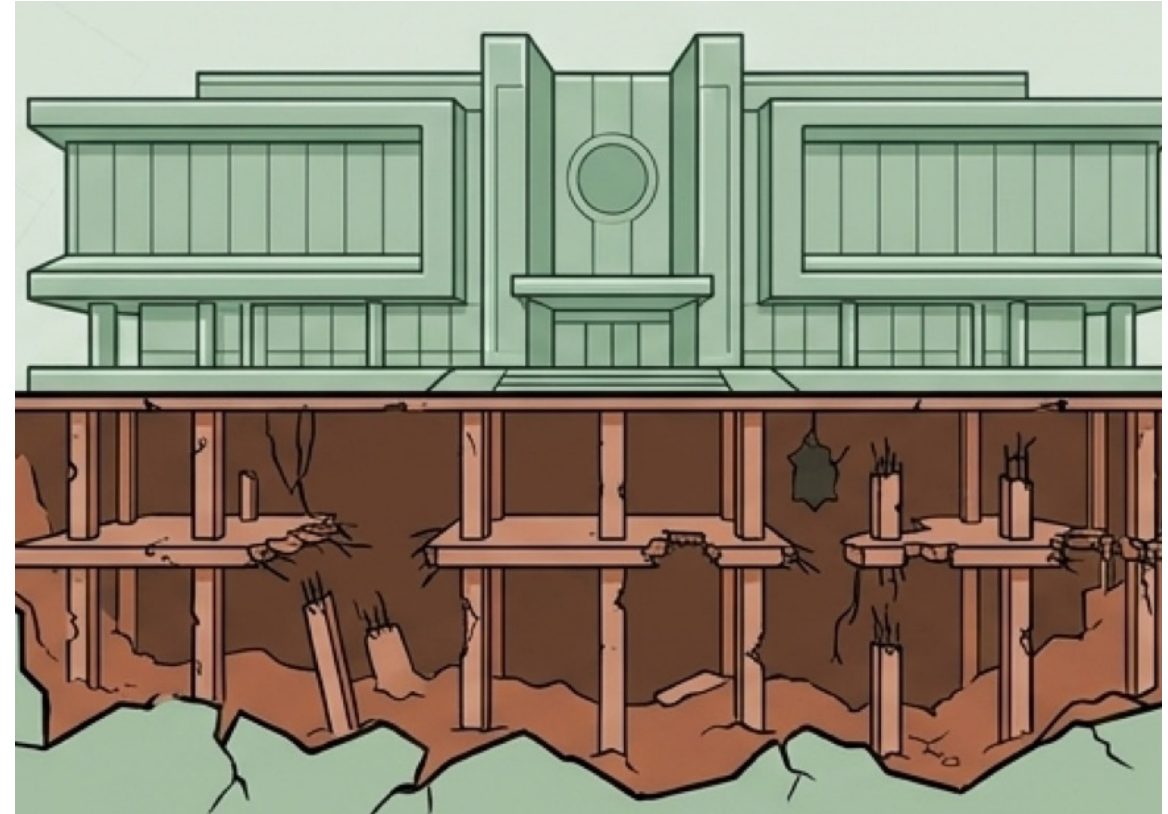
El aprendizaje supone tiempo y algo de frustración

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Riesgos actuales

- Aprendizaje vs eficiencia
 - Resolver rápidamente
 - Aprender a resolver
- Evitar la deuda técnica
 - Comprender los cimientos
 - PoCs no escalables
- El aprendizaje necesita tiempo

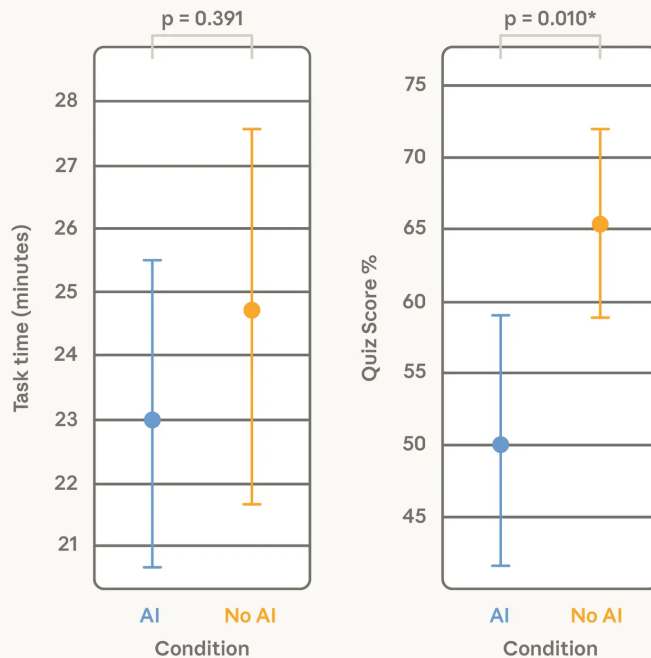


Riesgos actuales

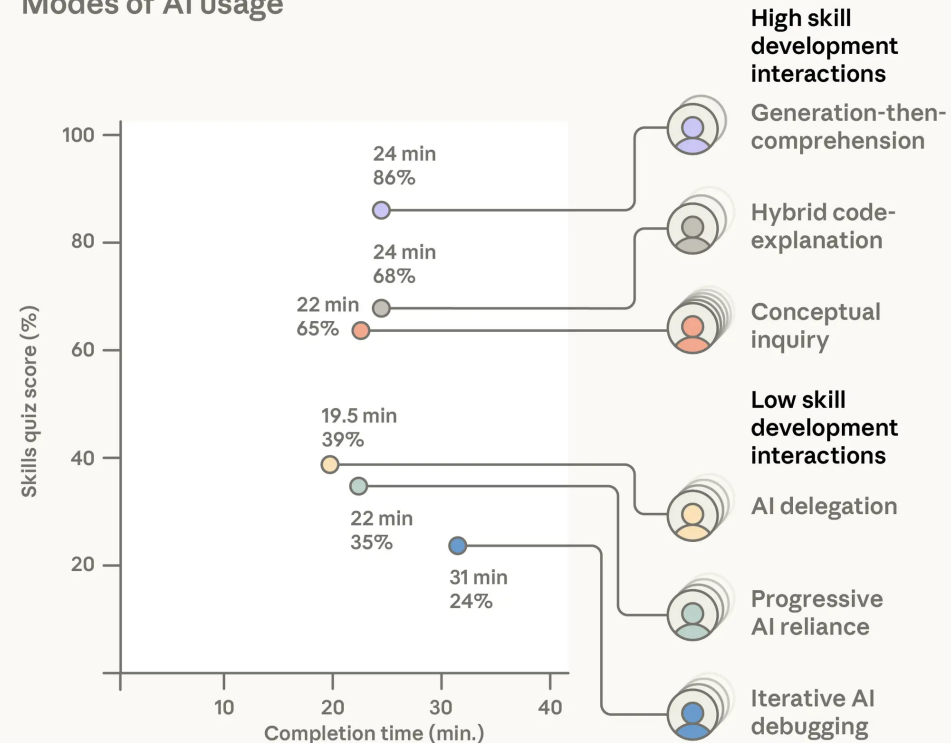
Enero 2026: <https://www.anthropic.com/research/AI-assistance-coding-skills>

How AI assistance impacts coding speed and skill formation

AI assistance: treatment effect on coding speed and knowledge score



Modes of AI usage



Un ejemplo a evitar

- StackOverflow vs ChatGPT:

